Chapitre 5

LES AUTOMATES D'ÉTAT FINI

1. Introduction

Les automates sont des objets mathématiques, très utilisés en informatique, qui permettent de modéliser un grand nombre de systèmes (informatiques). En compilation les automates servent à constituer des analyseurs lexicaux qui permettent de repérer les mots d'un langage. De façon très informelle, un automate est un ensemble "d'états du système", reliés entre eux par des "transitions" qui sont marquées par des symboles. Étant donné un "mot" fourni en entrée, l'automate lit les symboles du mot un par un et va d'état en état selon les transitions. Le mot lu est soit accepté par l'automate soit rejeté.

2. Automates finis

Définition

Un **automate fini** (**AFN**) est défini par un quintuplé (A, Q, I, F, μ) tel que :

A est **l'alphabet** (vocabulaire), ensemble fini, non vide de symboles ;

Q est **l'ensemble des états** possibles pour la machine (fini et non vide) ;

$I \subseteq Q$ est l'ensemble des états initiaux ou états de départ ;

$F \subseteq Q$ correspond aux états finaux ou états d'acceptation ;

μ la fonction de transition telle que :

$\mu : Q \times A \cup \{\varepsilon\} \rightarrow Q$.

Pour la configuration courante d'état et de symbole, $\mu(q_i, a_j) = q_k$ signifie que nous passons de l'état q_i vers l'état q_k en lisant le symbole a_j.

2.1 Fonctionnement d'un automate fini déterministe

L'automate prend en entrée un mot et l'accepte ou le rejette. Autrement dit, il le reconnaît ou ne le reconnaît pas. *Le langage associé à un automate est constitué de l'ensemble des mots qu'il reconnaît*. Voici comment l'automate procède pour décider si un mot appartient à son langage.

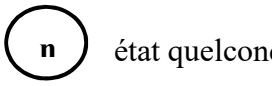
- Le processus commence à l'état de départ q_0.
- Les symboles du mot sont lus les uns après les autres.

• À la lecture de chaque symbole, la fonction de transition δ est employée pour se déplacer vers le prochain état (en utilisant l'état actuel et le caractère qui vient d'être lu).

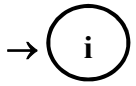
• le mot est reconnu si et seulement si le dernier état (*i.e.*, l'état correspondant à la lecture du dernier caractère du mot) est un état de *F*.

2.2. Représentation graphique

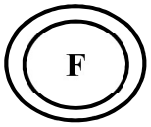
Un AFN peut être représenté par un graphe orienté, appelé "diagramme de transition" ou "graphe de transition". Les nœuds correspondent aux états et les arcs représentent les transitions possibles (calculées par la fonction de transition) entre les états. Ces arcs sont étiquetés par le symbole permettant cette transition. Les états initiaux sont représentés par un nœud possédant une flèche entrante sans origine. Un état final est représenté par un nœud doublement cerclé ou un nœud avec une flèche sortante.

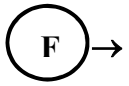


état quelconque

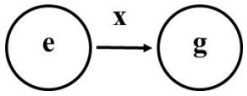


état initial





état final



est une transition où e est l'état de départ et f l'état d'arrivée ((x,e) → f). Cette notation est équivalente à μ(e, x) = f (ou δ(e, x) = f). Nous disons que l'automate passe de l'état **e** à l'état **f** en lisant le caractère **x**.

2.3. Représentation matricielle

Un automate peut être également caractérisé par une matrice de transition. En effet, la fonction de transition est très souvent représentée par une "table de transition" (matrice de transition) qui sont des tableaux (ou des matrices) dont les colonnes correspondent aux symboles et les lignes aux états. La cellule (l_i,c_j) indique les transitions (a_i, q_j) → q_k.

Exemple

L'automate T étudié précédemment peut être représenté par :

μ	**a**	**b**
→ 1	**1**	**2**
*** 2**	**1**	**2**

Il est à noter que dans la représentation matricielle, devant un état initial doit figurer une flèche et devant un état final une étoile.

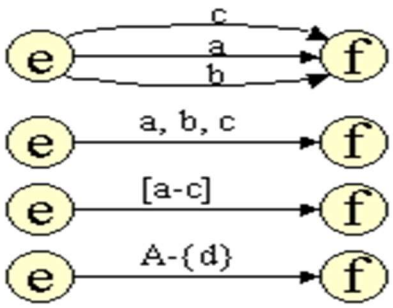
Définition

Le langage reconnu (ou accepté) par un automate fini non déterministe (AFN) est l'ensemble des mots acceptés par l'automate. Un mot w est reconnu (accepté) par un AFN si :

- Il existe un chemin menant d'un état initial à un état final, étiqueté par la suite des symboles de w.
- ε est neutre pour la concaténation.

Remarque

Dans certains cas, il est plus facile de représenter un groupe de transitions par une notation ensembliste. Par exemple sur l'alphabet A = {a, b, c, d}, supposons que pour un état donné e, nous avons un passage à l'état f pour les symboles a, b et c.



3. Propriétés des états

Outre les états finaux et les états initiaux, il existe d'autres catégories d'états spécifiques : les états stériles, accessibles, co-accessibles, utiles, inutiles et isolés.

3. 1. État stérile

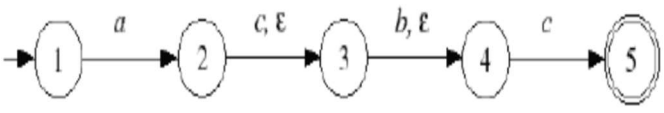
Un état stérile (état mort) est un état pour lequel il n'est pas possible d'atteindre un état final.

3. 2. État accessible

Un état q de A est dit accessible s'il existe un chemin partant de l'état initial et arrivant en q.

Remarque

Une ε-transition (notée ε sur l'arc d'un automate) permet de passer d'un état à l'autre d'un automate sans lire de symbole.



Le langage reconnu est L = {acbc, acc, abc, ac}

3. 3. État co-accessible

Un état q de A est dit co-accessible si de q part un chemin qui peut atteindre un état final.

3. 4. État utile

Un état q de A est dit utile s'il est à la fois accessible et co-accessible.

3. 5. État inutile

Un état inutile est soit un état qui n'est pas accessible, ou qui n'est pas co-accessible.

3. 6. État isolé

Un état e est dit **isolé** s'il ne participe à aucune transition.

4. Automate fini non déterministe

Un automate fini non-déterministe est un automate tel que dans un état donné, il peut y avoir plusieurs transitions avec le même symbole. Le fonctionnement d'un tel automate n'est donc pas totalement « déterminé », car on ne sait pas quel état l'automate va choisir.

5. Automate fini déterministe

Définition

Un automate fini déterministe (AFD) sur un alphabet A est un quadruplet (Q, δ, i_0, F) ssi :

1. δ est une fonction de transition : $\delta : Q \times A \rightarrow Q$.
2. avec un unique état initial.
3. à partir de chaque état, il y a au plus une transition d'étiquette donnée.
4. les ε-transitions ne sont pas autorisées.

5.1 Déterminisation d'un automate sans ε-transitions

Théorème

Soit T = (A, Q, I, F, δ) un AFN qui reconnaît le langage L = L(T) alors il existe un AFD T' = (A, Q', I', F', δ ') tel que L' = L(T') = L.

L'automate T'est construit de la façon suivante :

- Q' ⊆ P(Q), c'est-à-dire que les états de T' sont des ensembles d'états de T ;
- I' = {I} ;

- $F' = \{Q' \subseteq Q : Q' \cap F \neq \emptyset\}$;
- $\forall S \subset Q$ et $\forall a \in A$, $\delta'(a,S)=S'$ où $S'=\{p \mid \exists q \in S : p \in \delta(a,q)\}$.

5.1. Algorithme

1. Commencer avec l'état initial (ou ensemble des états initiaux)

2. Rajouter dans la table de transition tout les nouveaux "états" produits avec leurs transitions.

3. Recommencer en 2 jusqu'à ce qu'il n'y ait plus de nouvel "état" .

4. Tous les "états" contenant au moins un état terminal deviennent terminaux.

5. Renuméroter les états (facultatif)

Théorème

Si L et L' sont des langages reconnus par un AFN (respectivement un AFD), alors il en est de même :

- du complémentaire $A^* \setminus L$.
- de l'intersection $L \cap L'$.
- de l'union $L \cup L'$.
- de la concaténation LL'.
- de la mise à l'étoile L^*.

6. Minimisation d'un automate fini par raffinement successif

Certains AFD possèdent un nombre d'états et de transitions important. États et transitions ne sont pas toujours indispensables. Une première approche de cette minimisation peut consister à étudier les caractéristiques des états et de supprimer ceux qui sont inutiles.

Algorithme

Etape 0 :

$p, q \in Q$, $p \approx_0 q$ ssi

$(p \in F$ et $q \in F)$ ou $(p \notin F$ et $q \notin F)$

Etape i :

$\forall i>0$, $p \approx_i q$ ssi

$p \approx_{i-1} q$ et $\forall a \in A$, $\mu(p, a) \approx_{i-1} \mu(q, a)$

Etape finale (arrêt des itérations) :

$\approx_i = \approx_{i-1}$

Une division de l'ensemble des états en deux classes : les états terminaux et les autres.

Pour chaque lettre de l'alphabet, on examine si elle sépare deux états de la même classe, c'est-à-dire si sa transition envoie le premier dans une classe et le second dans une autre: si tel est le cas, on subdivise la classe en question.

On répète l'opération jusqu'à ce qu'aucune subdivision ne se produise plus.

La démarche

1) On construit un tableau de taille N × N (N le nombre d'états de l'automate). Chaque case dans ce tableau correspond à un couple d'états (p,q). Pour ne pas avoir deux fois le même couple ((p,q) et (q,p)) ni les couples inutiles (p,p), on garde seulement la partie triangulaire supérieure (ou inférieure) du tableau. Notre but final est de marquer toute les cases (p,q) pour des états p et q non-équivalents.

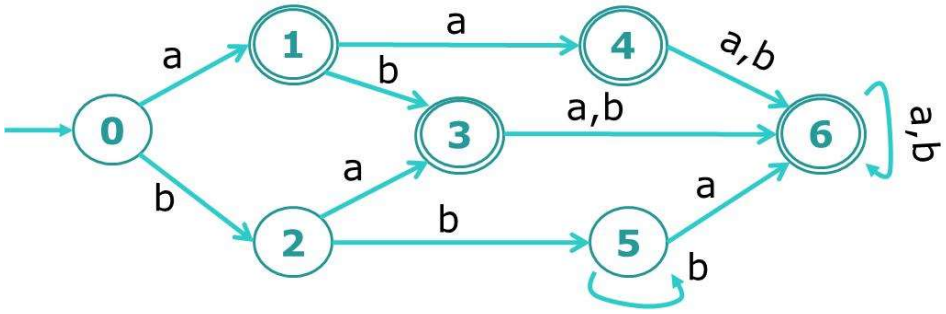
2) Au début on marque chaque case (p,q) correspondant à un état final et à un état non-final (c-à-d $p \notin F$ et $q \in F$ ou à l'envers, $p \in F$ et $q \notin F$).

3) On parcourt toutes les cases vides du tableau d'une manière quelconque en le modifiant de façon suivante : pour chaque case (p,q) et lettre a, on calcule $\mu(p,a) = p_0$ et $\mu(q,a) = q_0$.

On marque (p,q) si et seulement si la case (p_0,q_0) est déjà marquée. Le balayage est répété jusqu'au moment quand le tableau ne se modifie plus.

4) La relation d'équivalence se cache dans le tableau final : deux états p et q sont équivalents si et seulement si la case (p,q) n'est pas marquée.

Exemple



A = {0, 2, 5} B = {1, 3, 4, 6}

0							
1	X						
2		X					
3	X		X				
4	X		X				
5		X		X	X		
6	X		X			X	
	0	1	2	3	4	5	6

Initialisation X ≈ Itération N° 1



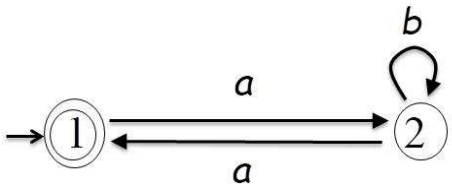
7. Automates finis et expressions régulières

Soit T = {A, Q, I, F, μ} un automate fini. Pour p ∈ Q, X_p est le langage des mots acceptés avec p comme état initial. Pour tout état X_p nous avons :

$$Xp = \begin{cases} \sum_{(p,a,q)\in\mu} aXq + \varepsilon & si\ p\ \in F \\ \sum_{(p,a,q)\in\mu} aXq & sinon \end{cases}$$

En appliquant cette équation pour chacun des états de l'automate, nous obtenons alors un système de n équations rationnelles (n = |Q|) à n inconnues où les inconnues sont les Xp et les coefficients sont des expressions rationnelles. La résolution d'un tel système s'effectue en utilisant principalement le lemme d'Arden. Le langage reconnu par l'automate est celui de l'état initial.

Exemple



$L_1 = aL_2 + 1$ (car l'état 1 est final)

$L_2 = bL_2 + aL_1$

$L_2 = b*aL_1$

$L_1 = ab*aL_1 + 1 = (ab*a)*$

L(T) est décrit par (ab*a)*.